ARCHITECTURE OF 8086 MICROPROCESSOR

✅ General Information

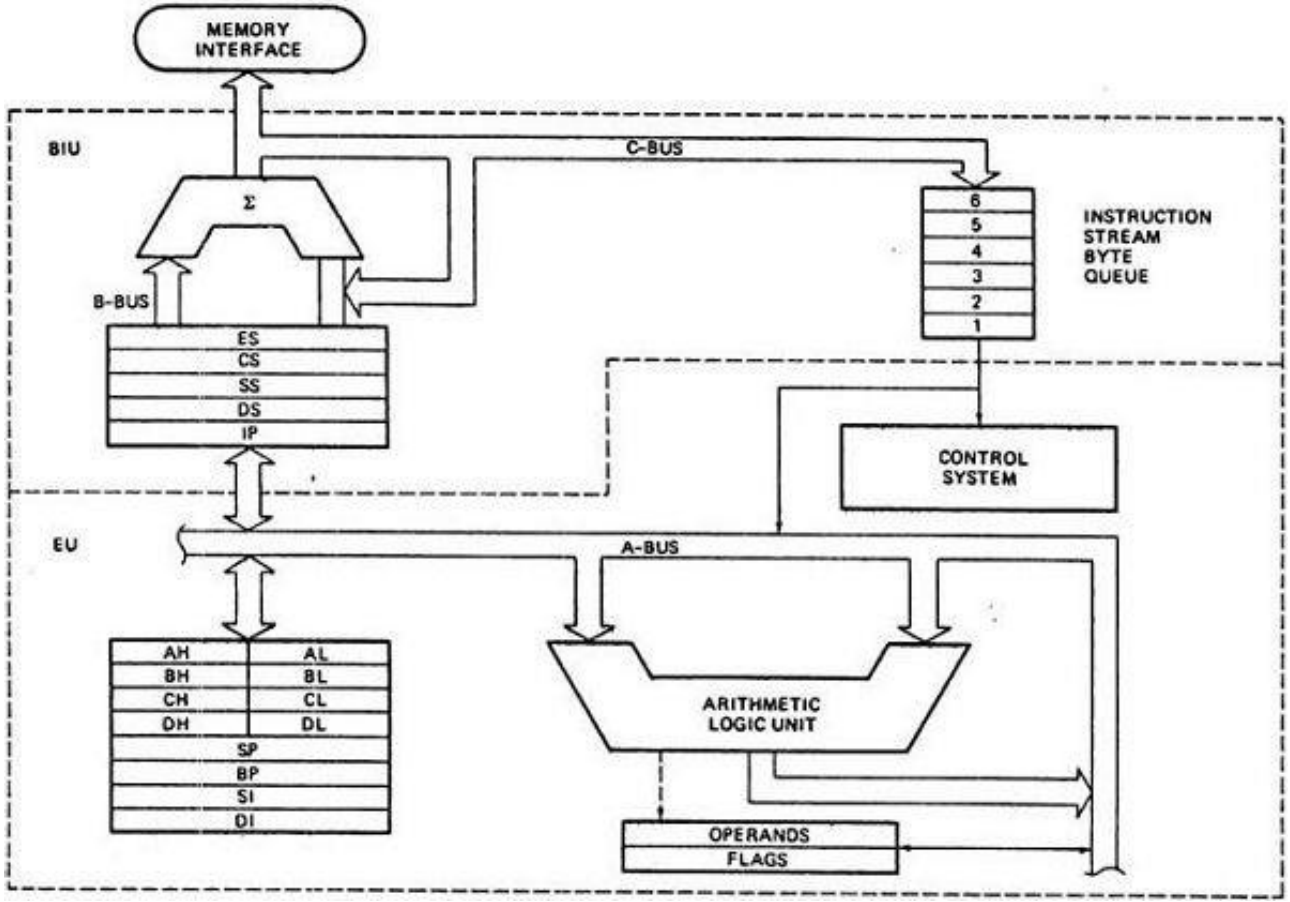
1. **Launch Year**: 1978
2. **Bit Size**: First **16-bit** microprocessor.
3. **Transistors**: Contains **29,000** transistors.
4. **Package Type**: Available in a **40-pin Dual-Inline Package (DIP)**.
5. **Improvement**: Major speed and architecture improvement over **Intel 8085**.

✅ Available Versions

- **8086** – 5 MHz
- **8086-2** – 8 MHz
- **8086-1** – 10 MHz

- **16-bit Processor**: Data bus width is 16 bits.
- **20-bit Address Bus**: Can access **2^{20} = 1 MB** memory (from 00000H to FFFFFH).
- **Prefetch Queue**: Can **fetch 6 instruction bytes** ahead to speed up execution (pipelining).
- **Clock Requirement**: Needs a **single-phase clock with 33% duty cycle**.
- **Power Supply**: Operates on a **+5V power supply**.

ARCHITECTURE OF 8086 MICROPROCESSOR

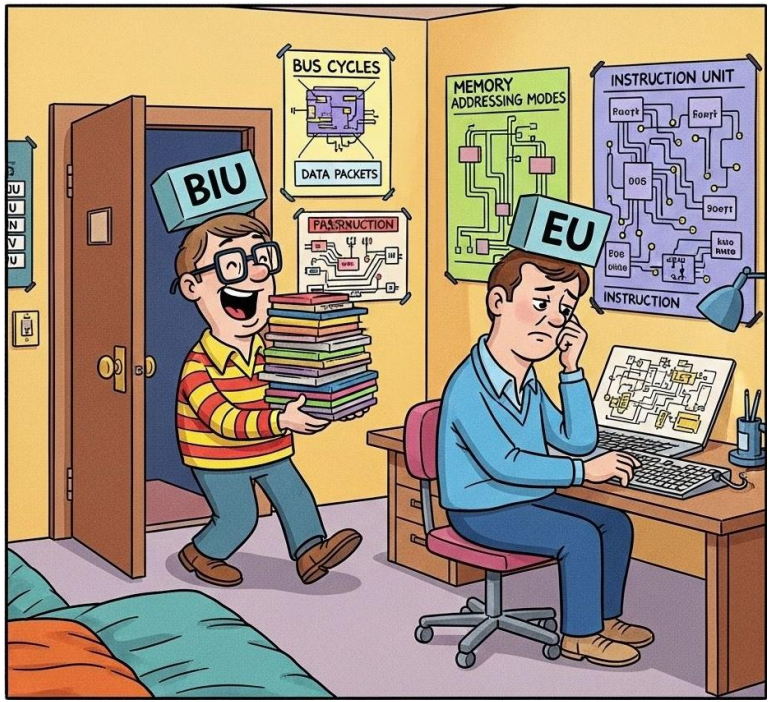


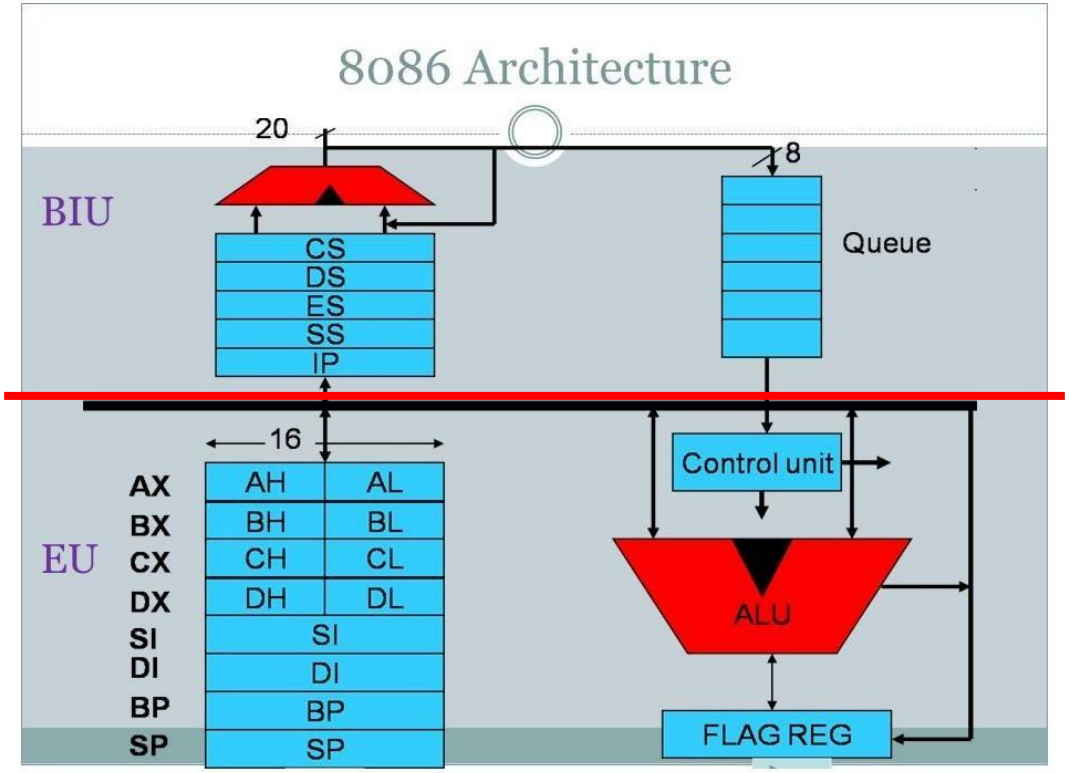
BEFORE

	1	2	3	4	5	6
I_1	F	D	E			
I_2				F	D	E

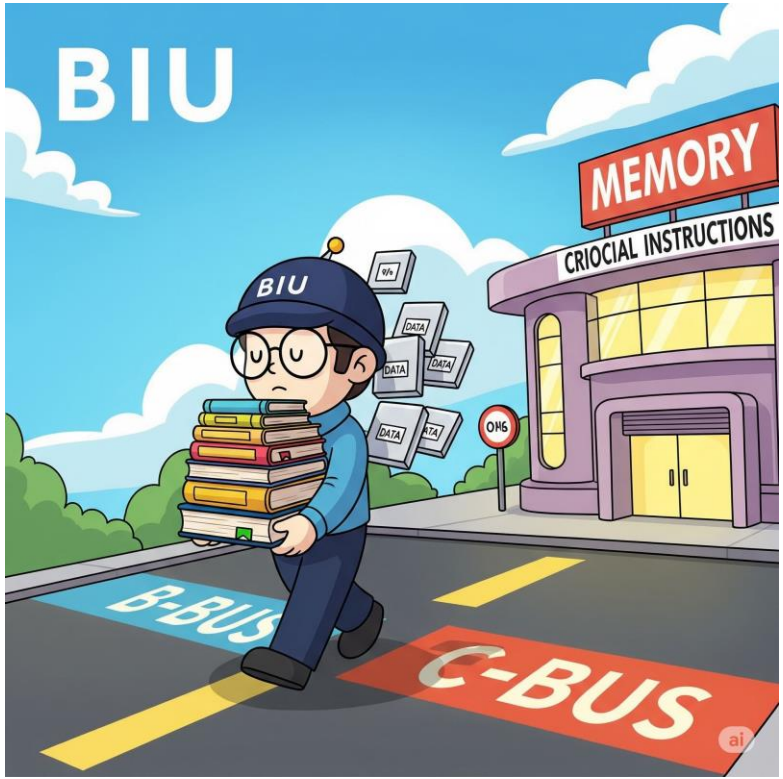
AFTER

	1	2	3	4	5	6
I_1	F	D	E			
I_2		F	D	E		





Bus Interface Unit (BIU) & Execution Unit (EU)



Bus Interface Unit (BIU):

The Bus Interface Unit (BIU) is responsible for communicating with memory and I/O devices. It acts like the messenger of the CPU, managing data flow between the processor and external systems. It handles:

- **Fetching instructions from memory.**
- **Reading/writing data to and from memory or I/O ports.**
- **Calculating physical addresses using segment registers and offset.**
- **Queuing instructions using a 6-byte prefetch queue to speed up execution.**
- **Controlling the system bus (address and data bus).**

Execution Unit (EU):

The Execution Unit (EU) is the part that actually executes the instructions fetched by the BIU. It includes:

- **Arithmetic Logic Unit (ALU) – performs calculations (add, sub, etc.).**
- **Control unit – decodes instructions and controls execution.**
- **Registers – stores temporary data, like AX, BX, CX, DX, etc.**
- **Flag register – holds status flags (zero, carry, sign, etc.).**

ARCHITECTURE OF 8086 MICROPROCESSOR

1. A-Bus (Address Bus)

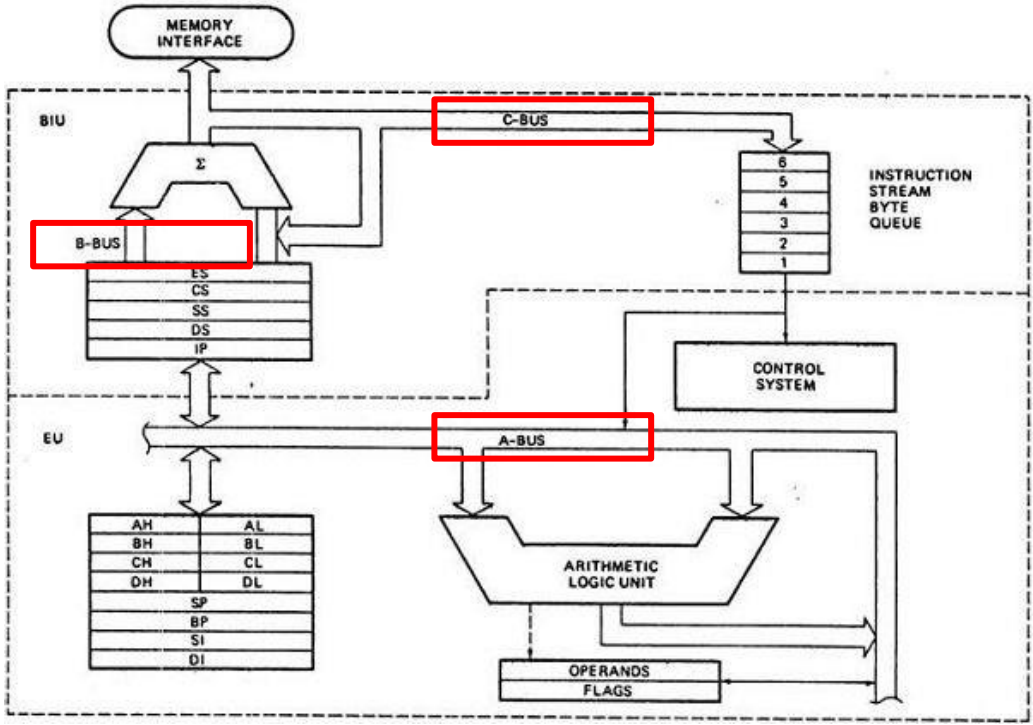
- Carries address information.
- Used internally for effective address calculation.
- Transfers addresses between registers and address generation logic.

2. B-Bus (Data Bus)

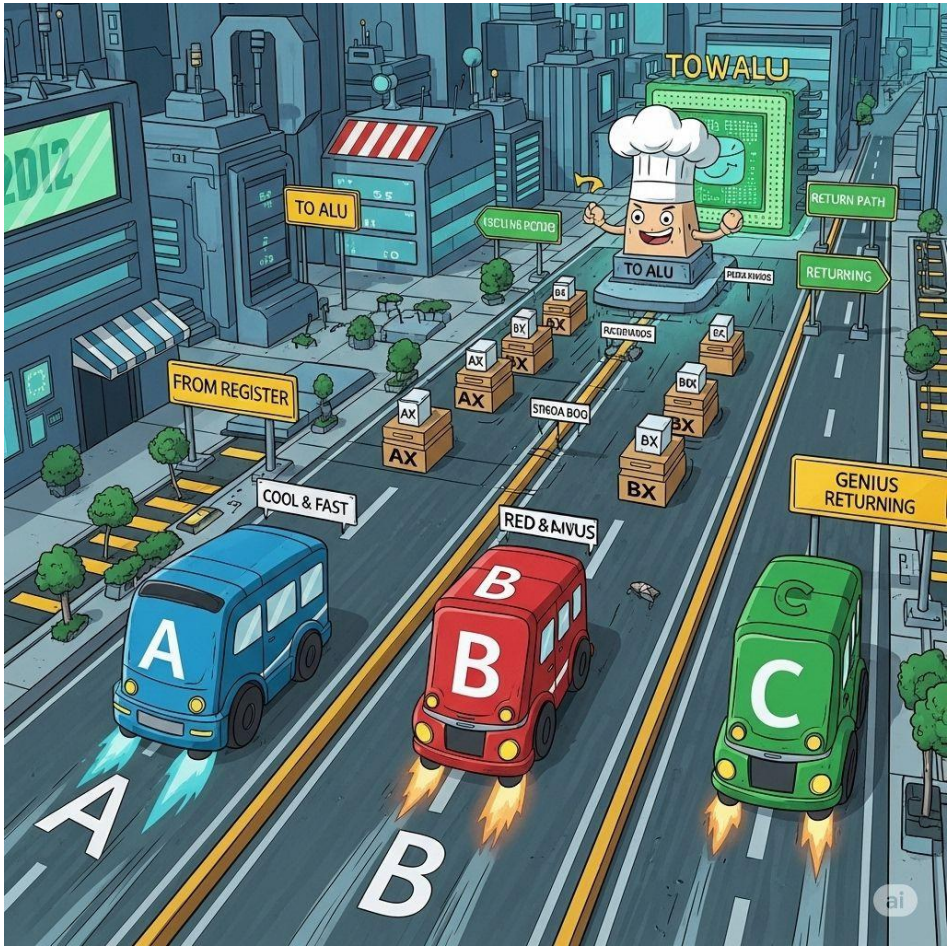
- Carries operand/data to the ALU.
- Connects general-purpose registers to the ALU.
- Used during arithmetic and logical operations.

3. C-Bus (Control Bus)

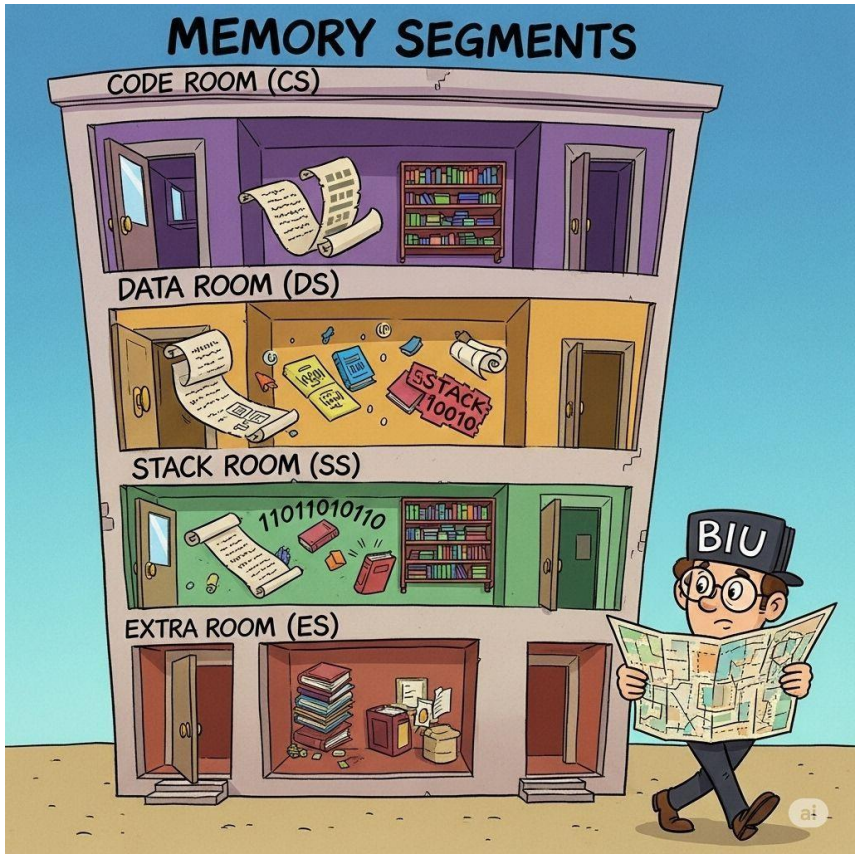
- Carries control signals.
- Tells internal components what operation to perform (e.g., read/write, ALU commands).
- Controls data flow and operations within the processor.



িতন বাে সর শহরভ্রমণ

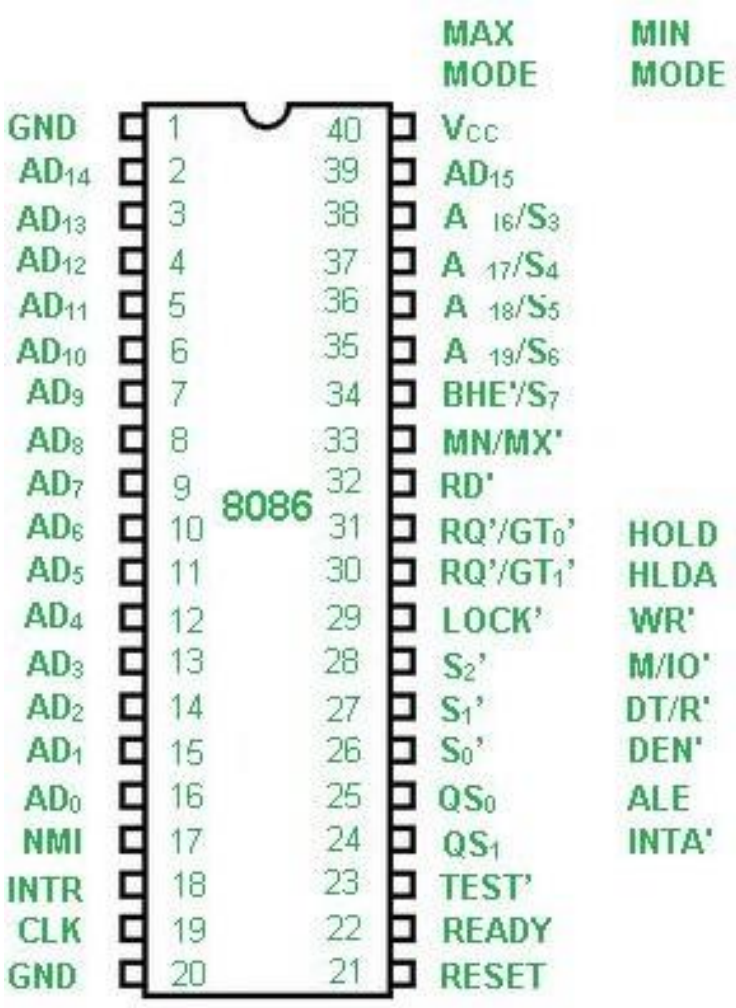


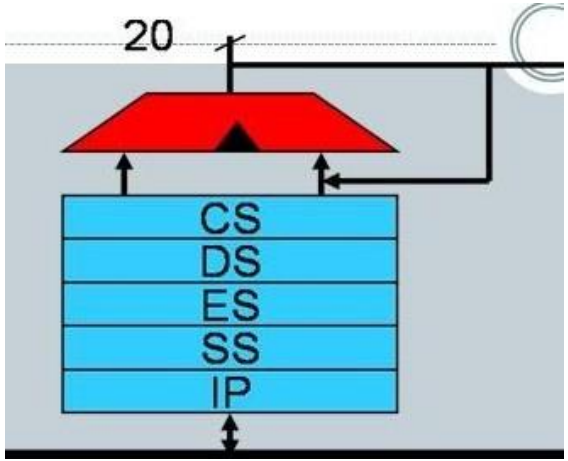
BIU বাবুর স্টাক বাড়ি সফর



🧰 Advantages of Segmentation

- Logical organization of code, data, and stack.
- Security and protection (in more advanced CPUs).
- Enables modular program design.
- Efficient memory utilization.


MAX MODE
MIN MODE
GND 1
AD14 2
AD13 3
AD12 4
AD11 5
AD10 6
AD9 7
AD8 8
AD7 9
AD6 10
AD5 11
AD4 12
AD3 13
AD2 14
AD1 15
AD0 16
NMI 17
INTR 18
CLK 19
GND 20
8086
40 VCC
39 AD15
38 A16/S3
37 A17/S4
36 A18/S5
35 A19/S6
34 BHE'/S7
33 MN/MX'
32 RD'
31 RQ'/GT0' HOLD
30 RQ'/GT1' HLDA
29 LOCK' WR'
28 S2' M/IO'
27 S1' DT/R'
26 S0' DEN'
25 QS0 ALE
24 QS1 INTA'
23 TEST'
22 READY
21 RESET



S4 (A17)	S3 (A16)	Segment
0	0	Extra Segment (ES)
0	1	Stack Segment (SS)
1	0	Code Segment (CS)
1	1	Data Segment (DS)

CS (Code Segment):

- Works with the **IP (Instruction Pointer)** to fetch instructions.
- `Physical Address = CS × 10H + IP`

DS (Data Segment):

- Works with general-purpose registers (e.g., AX, BX, etc.)
- Example: `MOV AX, [BX]` uses DS as the segment base by default.

SS (Stack Segment):

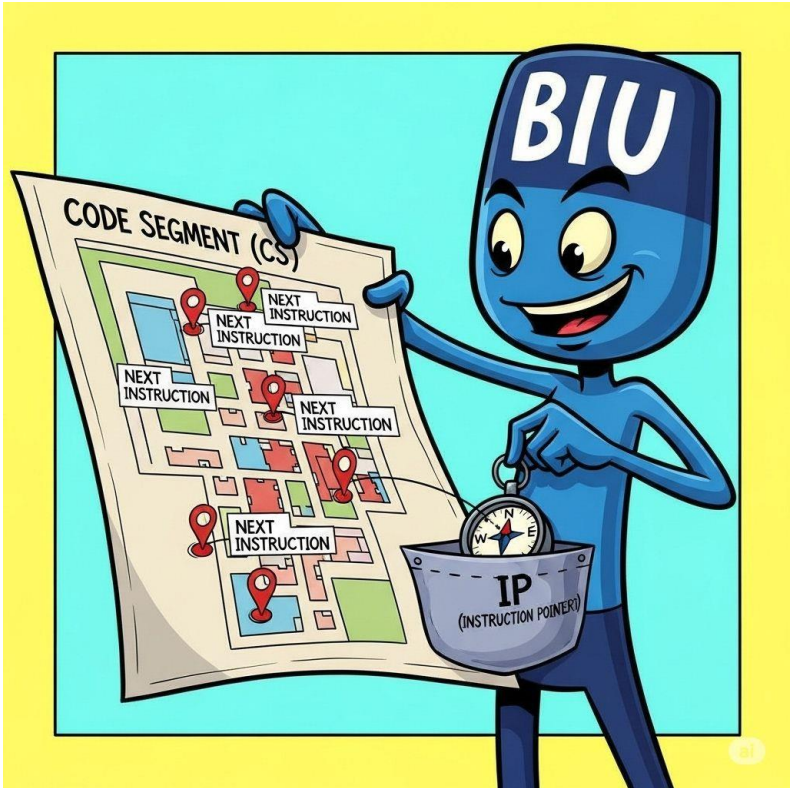
- Works with **SP (Stack Pointer)** and **BP (Base Pointer)**.
- Used during **PUSH, POP, CALL, RET**.

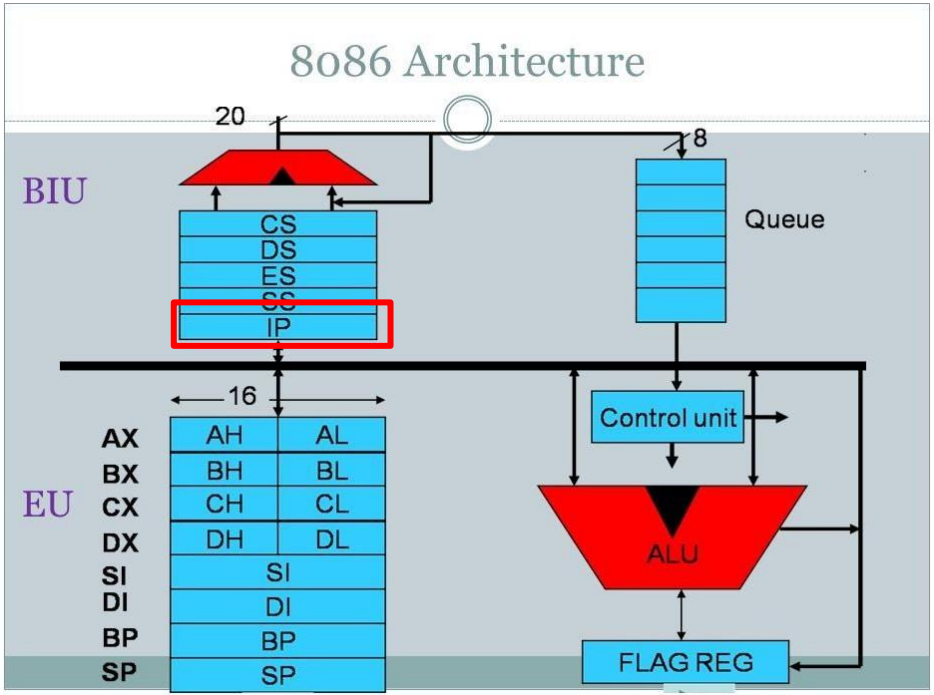
ES (Extra Segment):

- Used in **string operations**, especially with **DI (Destination Index)**.
- Example: `MOVSB`, `MOVSW` use ES:DI and DS:SI.

IP (Instruction Pointer) is a 16-bit register that holds the offset of the next instruction within the Code Segment (CS).

The physical address is calculated as: **(CS × 10H) + IP**.





What is Physical Address?

- The **actual 20-bit memory address** used by the 8086 microprocessor to access memory.
- Calculated using **Segment Register** and **Offset**.

📌 **Formula:**

🧮 **Physical Address = (Segment × 10H) + Offset**

- `Segment`: The base address (e.g., CS, DS, SS, ES)
- `10H` = 16 (to shift segment left by 4 bits)
- `Offset`: Distance from the start of the segment

Example:

- **CS = 1234H , IP = 0020H**

Physical Address = (1234H × 10H) + 0020H
= 12340H + 0020H
= 12360H

Offset

Offset is the distance (in bytes) from the beginning of a memory segment to a specific data or instruction location within that segment.

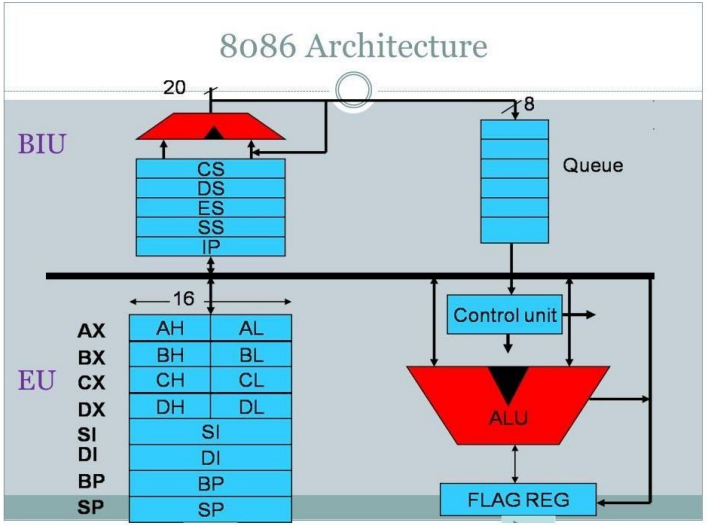
It is a **16-bit value** used in combination with a **segment register** to calculate the **physical memory address** in the 8086 microprocessor.

🧮 **Physical Address = (Segment × 10H) + Offset**

236 km (segment)

58.1 km (offset)

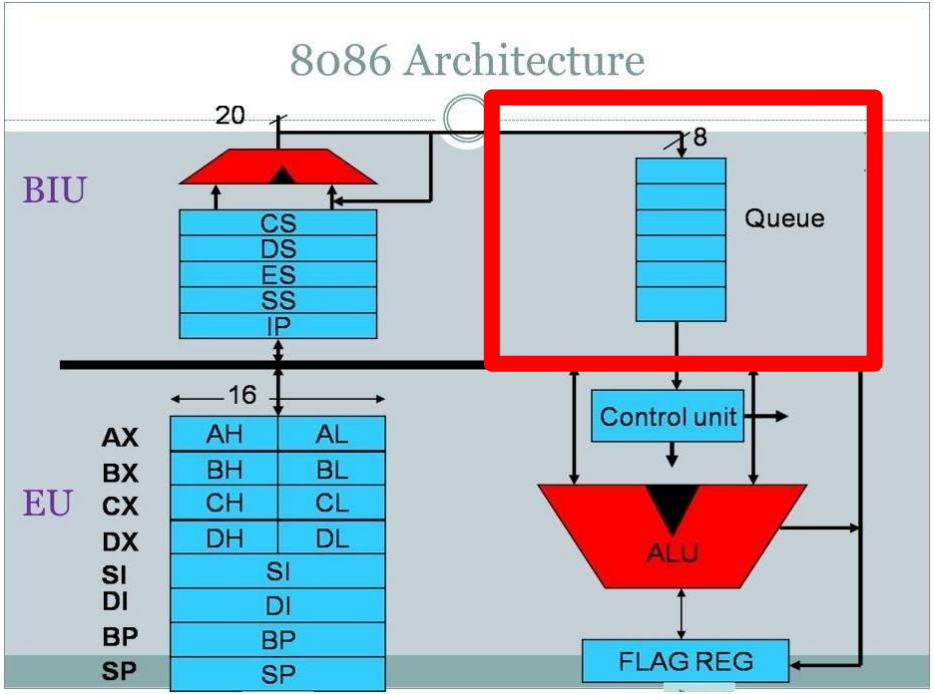
Segment Register	Used For	Offset Comes From
CS (Code Segment)	Executing instructions	**IP (Instruction Pointer)**
DS (Data Segment)	Accessing data	AX, BX, CX, DX, SI, DI, BP (typically SI/DI)
SS (Stack Segment)	Stack operations	**SP (Stack Pointer)** / BP
ES (Extra Segment)	String operations, extra data	**DI (Destination Index)**



BiU এর রান্নাঘর

Instruction Queue :

An **Instruction Queue** is a temporary storage in the CPU where instructions are pre-fetched and held before execution to improve processing speed and efficiency.



- **AX (Accumulator Register)**

Used for arithmetic, logic, and I/O operations. Often the default register in many instructions.

- **BX (Base Register)**

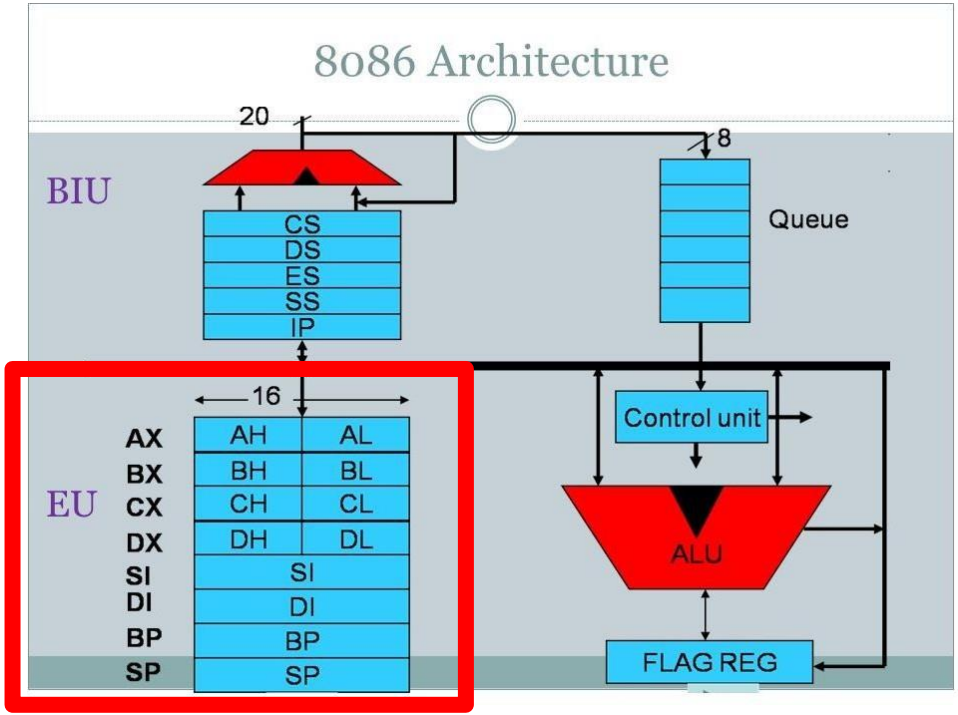
Used to hold base addresses for memory access. Helps in indirect addressing.

- **CX (Count Register)**

Acts as a loop counter in instructions like `LOOP`, `REP`, etc.

- **DX (Data Register)**

Holds data for I/O and used in multiplication/division with AX.



- **SI (Source Index)**

Points to source data in string operations. Works with DS segment.

- **DI (Destination Index)**

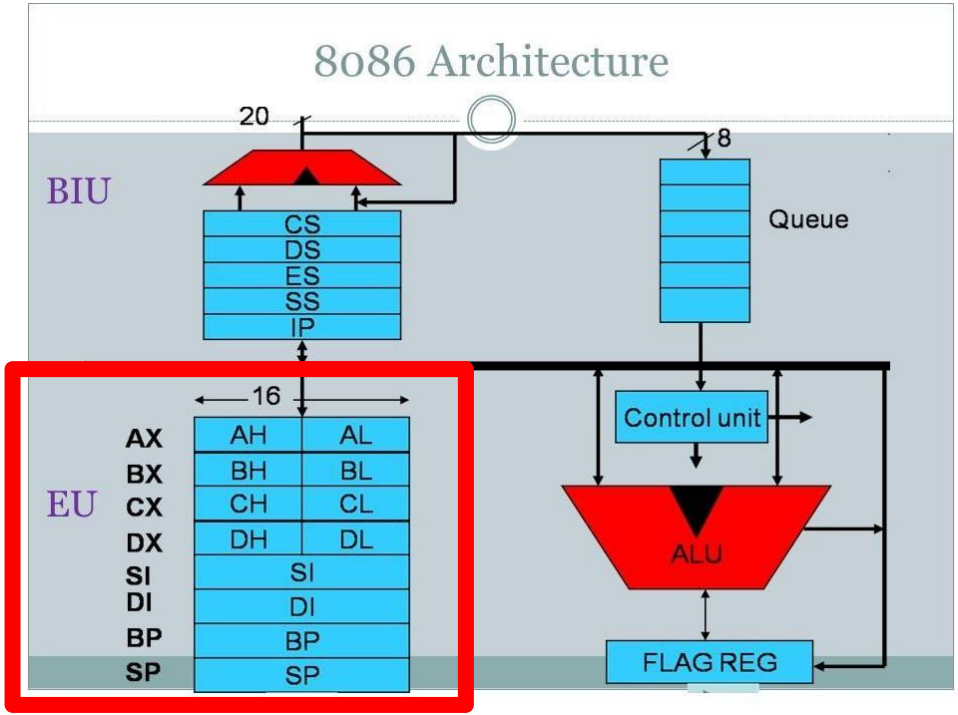
Points to destination in string operations. Works with ES segment.

- **SP (Stack Pointer)**

Points to the top of the stack. Used in `PUSH`, `POP`, `CALL`, and `RET`.

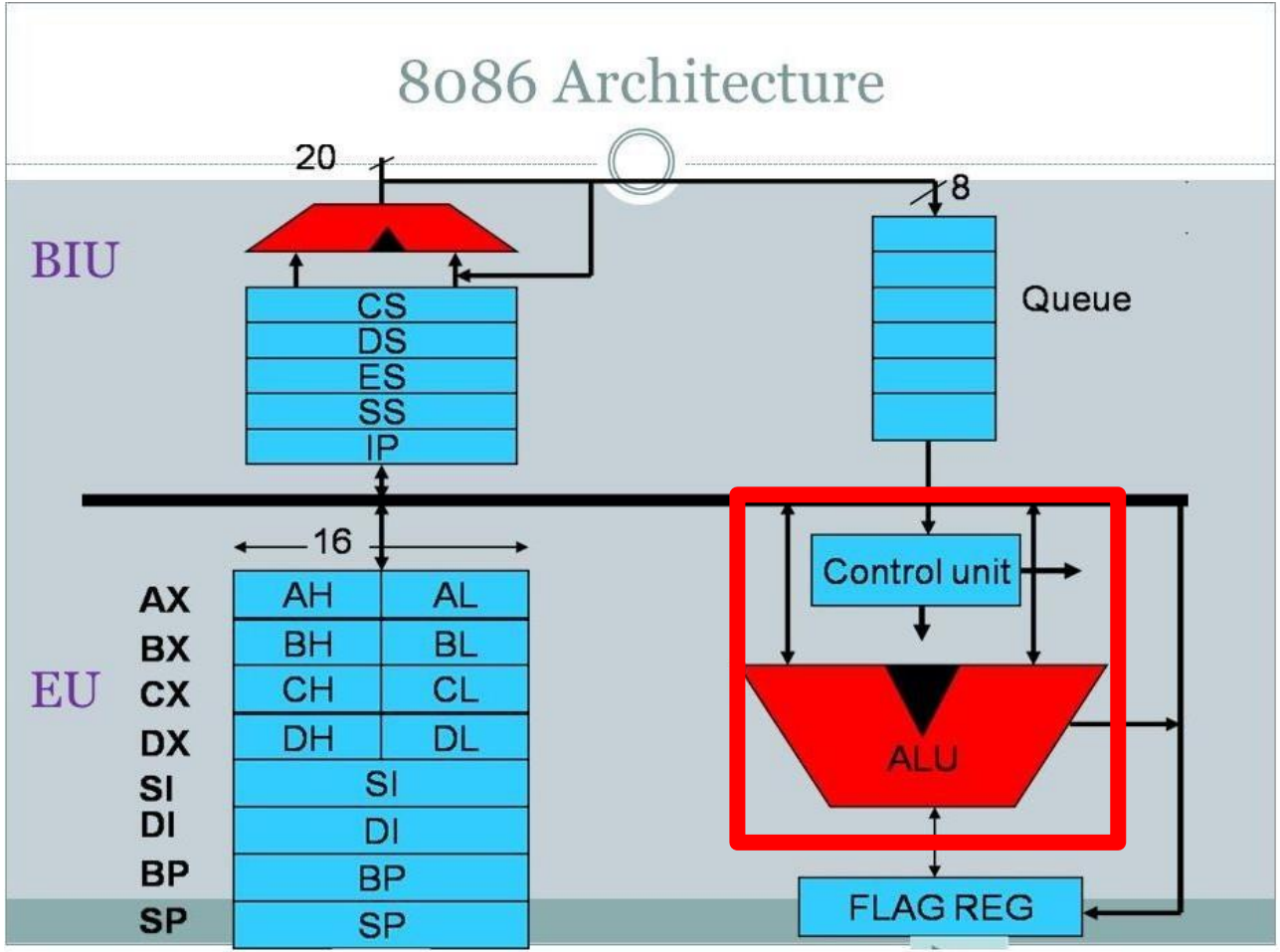
- **BP (Base Pointer)**

Accesses data in the stack frame. Used in function calls and local variable access.

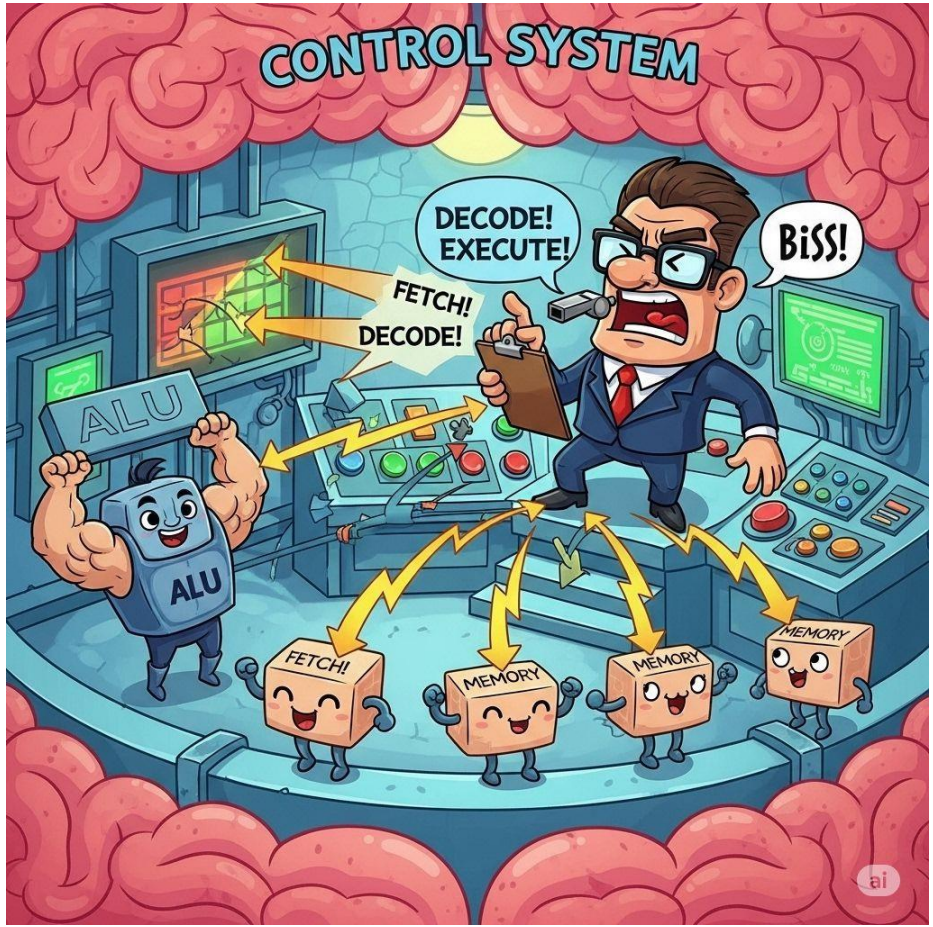


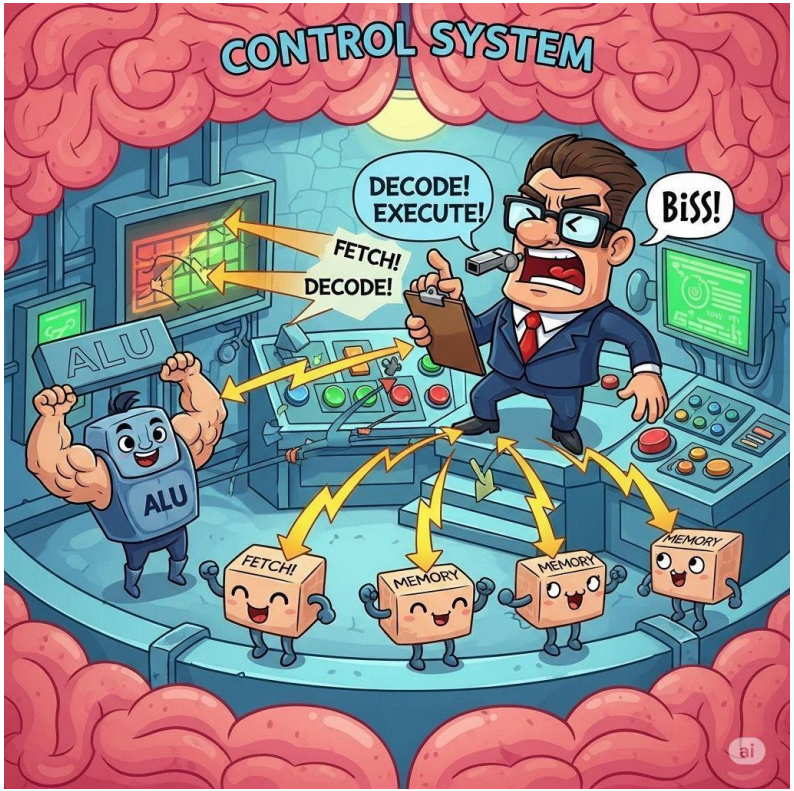

Register Set
AH
AL
AL
AL
SP
BH
CH
DH
DI
SI

8086 Architecture



স্কুলের করা স্যার আর আলাউিAন





Control Unit (CU):

Definition:

The **Control Unit** manages and coordinates the execution of instructions by directing the operation of the ALU, registers, and memory.

Function in 8086:

- Decodes instructions from the **Instruction Queue**.
- Sends control signals to **ALU** and other components.
- Ensures proper sequencing of operations.

অঙ্কের মাস্তান ALU

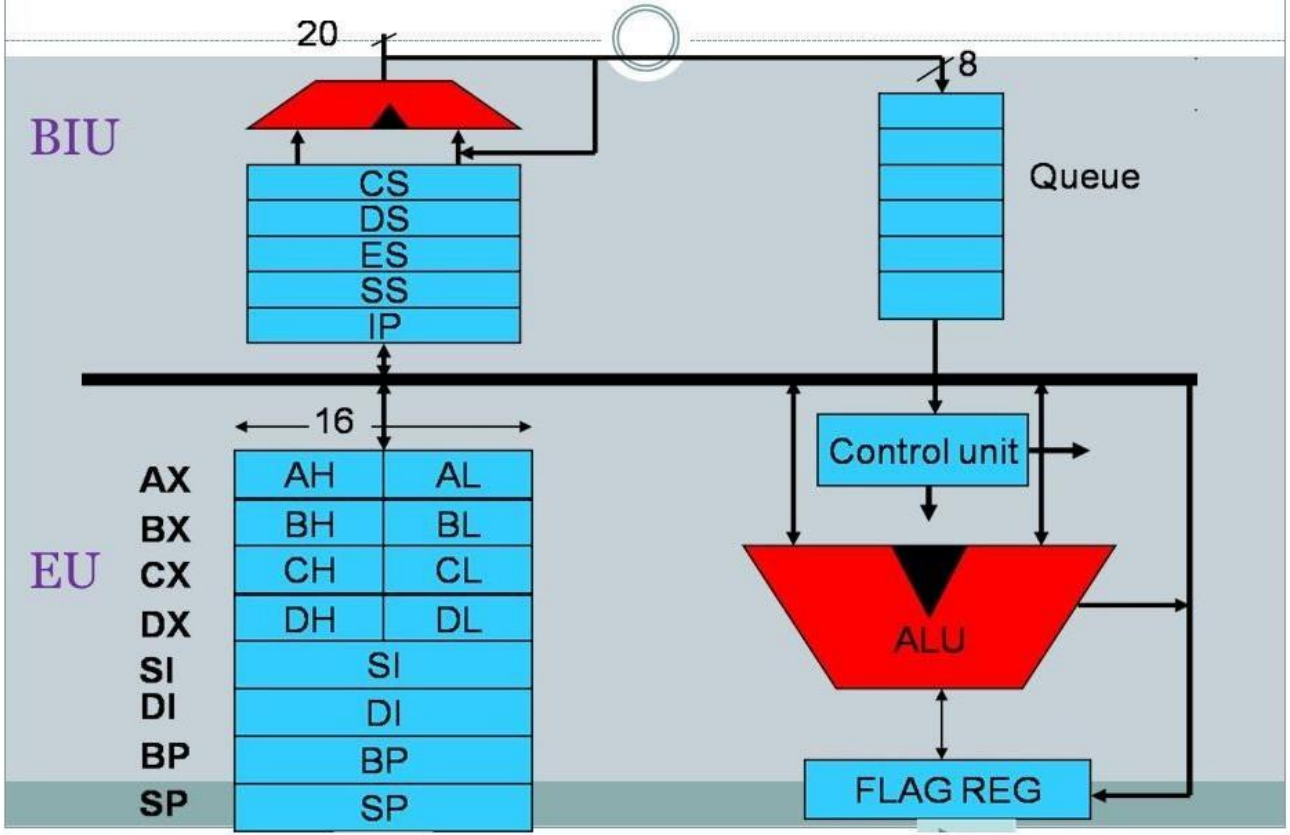
ALU (Arithmetic Logic Unit):

Definition:
The **ALU** performs all **arithmetic** (like addition, subtraction) and **logical** (like AND, OR, NOT) operations in the microprocessor.

Function in 8086:

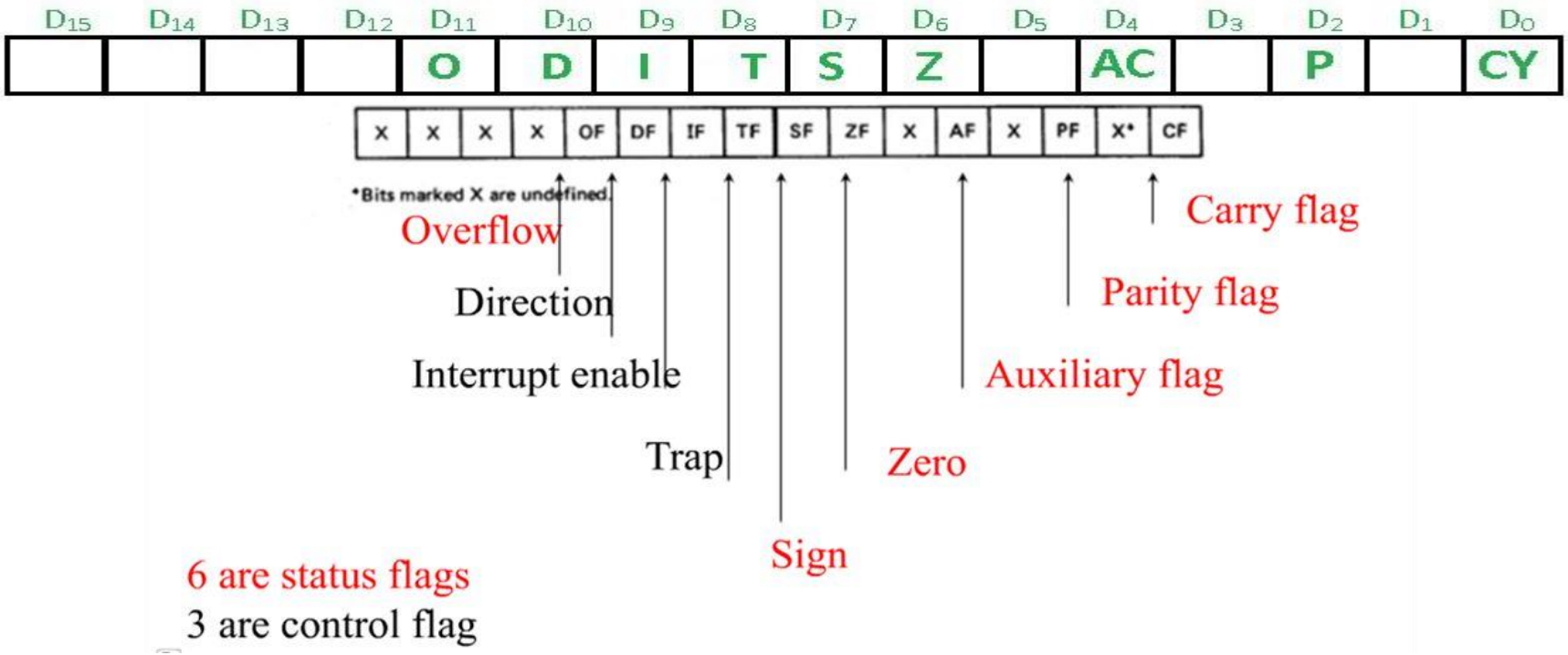
- Executes operations based on instructions from the Control Unit.
- Affects **flags** in the **Flag Register** based on results (e.g., Zero, Carry).

8086 Architecture



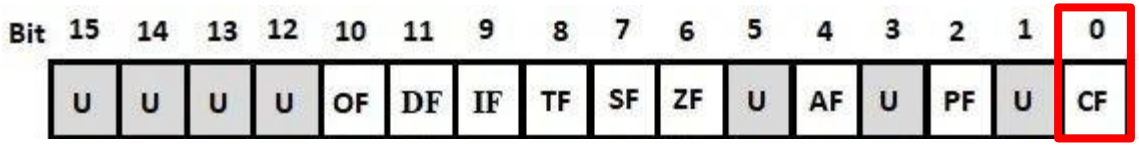
CPU-র মুড সুইং!

Flags



6 are status flags

3 are control flag



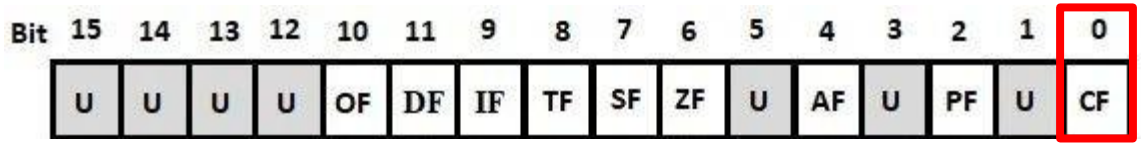
Carry Flag (CF)

- **Type:** Status Flag
- **Position:** D0
- **Definition:** Indicates an unsigned overflow, i.e., a carry out of the MSB in addition or a borrow in subtraction.
- **Affected by:** ADD, SUB, ADC, SBB, etc.

Example:

```
1111 1111 (255)
+
0000 0001 (1) = 1 0000 0000
            (Overflow) → CF = 1
```

- **Use Case:** Useful in multi-byte arithmetic or loop carry propagation.



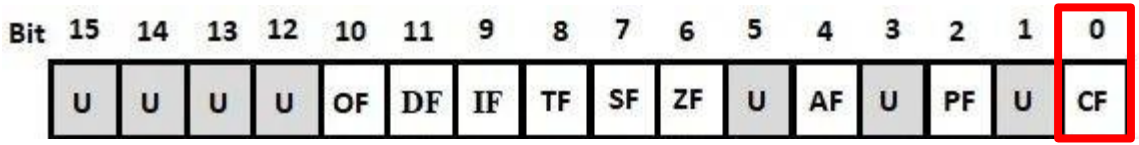
Parity Flag (PF)

- **Type:** Status Flag
- **Position:** D2
- **Definition:** Shows whether the number of 1s in the result is even.
- **Affected by:** Most arithmetic/logical operations

Example:

```
Result: 0000 1111 → Has four 1s → PF = 1 (Even)
Result: 0000 1011 → Has three 1s → PF = 0 (Odd)
```

- **Use Case:** Error checking or communication protocols.



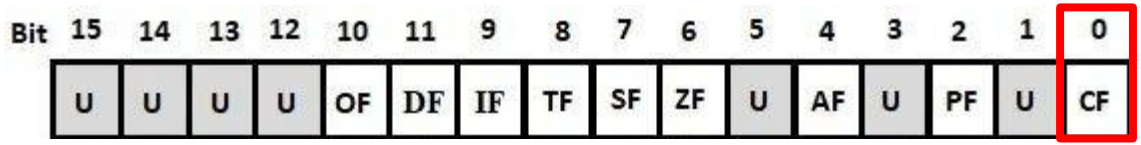
Auxiliary Carry Flag (AF)

- **Type:** Status Flag
- **Position:** D4
- **Definition:** Indicates a carry from bit 3 to bit 4. Used mainly in BCD operations.
- **Affected by:** ADD, SUB

Example:

```
1111 1111 (255)
+
0000 0001 (1) = 1 0000 0000
             AF = 1
```

- **Use Case:** Decimal (BCD) addition/correction.



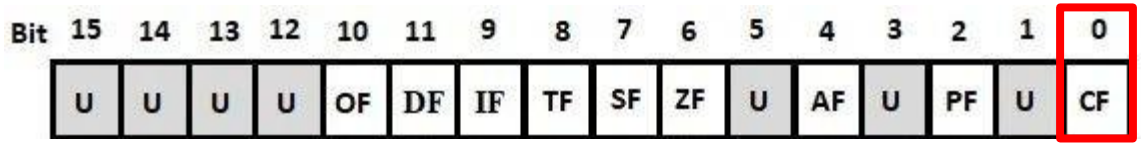
Zero Flag (ZF)

- **Type:** Status Flag
- **Position:** D6
- **Definition:** Indicates whether the result of an operation is zero.
- **Affected by:** CMP, SUB, AND, OR, XOR

Example:

```
0011 0000 - 0011 0000 = 0000 0000 → ZF = 1
```

- **Use Case:** Conditional execution, loops (e.g., JZ, JNZ).



Sign Flag (SF)

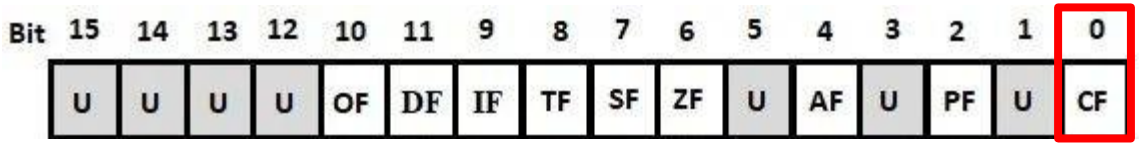
- **Type:** Status Flag
- **Position:** D7
- **Definition:** Reflects the sign (most significant bit) of the result.
- **Affected by:** Arithmetic and logical operations.

Example:

Result : `1000 0000 → SF = 1 (Negative Number)`

Result : `0100 1100 → SF = 0 (positive Number)`

- **Use Case:** Signed arithmetic and branching (e.g., JS, JNS).



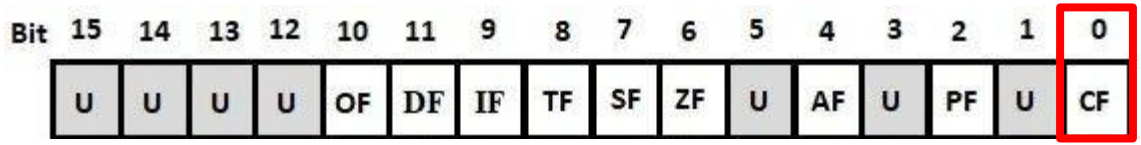
Trap Flag (TF)

- **Type:** Control Flag
- **Position:** D8
- **Definition:** Enables single-step debugging. When set, the processor interrupts after every instruction.

Example:

```
TF = 1 → Executes one instruction, then calls INT 1
```

- **Use Case:** Debuggers for instruction-by-instruction execution.



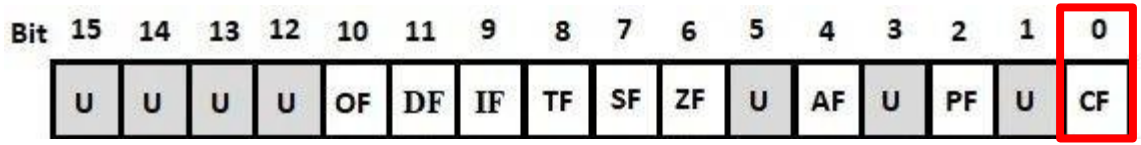
Interrupt Flag (IF)

- **Type:** Control Flag
- **Position:** D9
- **Definition:** Controls whether the processor will respond to maskable external interrupts.

Example:

```
IF = 1 → Interrupts are enabled (STI)
IF = 0 → Interrupts are disabled (CLI)
```

- **Use Case:** Critical code sections that shouldn't be interrupted.



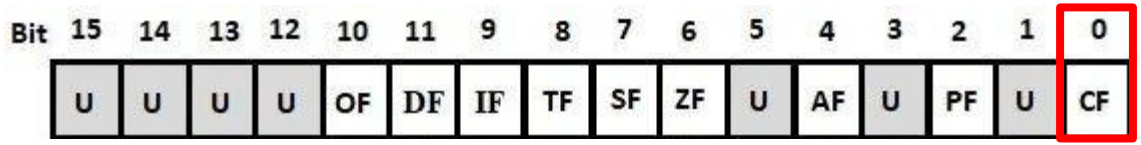
Direction Flag (DF)

- **Type:** Control Flag
- **Position:** D10
- **Definition:** Controls string processing direction:
  - `0` = Auto-increment (forward)
  - `1` = Auto-decrement (backward)

Example:

```
DF = 1 → Used in REP MOVSB to copy strings in reverse
```

- **Use Case:** `MOVS`, `CMPS`, `SCAS`, etc. for block operations.



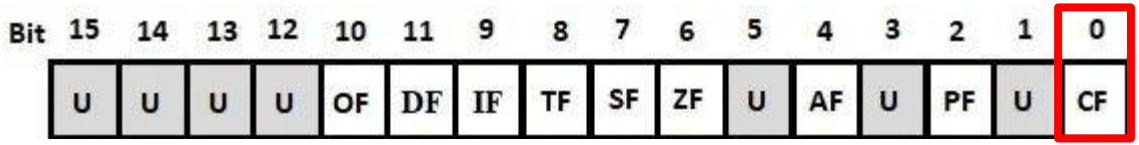
Overflow Flag (OF)

- **Type:** Status Flag
- **Position:** D11
- **Definition:** Indicates a signed overflow — when result is too large for destination in signed operations.

Example:

```
0111 1111 (+127)
+
0000 0001 (+1) = 1000 0000 → OF = 1
(Result looks like -128 in signed 8-bit!)
```

- **Use Case:** Signed comparison, conditional branching (JO, JNO)



❌ Unused/Reserved Bits (D1, D3, D5, D12–D15)

- These bits are reserved for internal or future use and should be ignored in 8086 programming.

🔹 Binary Addition Laws

- `0 + 0 = 0` (Carry 0)
- `0 + 1 = 1` (Carry 0)
- `1 + 0 = 1` (Carry 0)
- `1 + 1 = 0` (Carry 1)

📌 Carry is generated when both bits are 1.

🔹 Binary Subtraction Laws

- `0 - 0 = 0` (Borrow 0)
- `1 - 0 = 1` (Borrow 0)
- `1 - 1 = 0` (Borrow 0)
- `0 - 1 = 1` (Borrow 1)

📌 Borrow happens when subtracting 1 from 0.

Decimal **10** = Hexadecimal **A**

Decimal **11** = Hexadecimal **B**

Decimal **12** = Hexadecimal **C**

Decimal **13** = Hexadecimal **D**

Decimal **14** = Hexadecimal **E**

Decimal **15** = Hexadecimal **F**

Type	Decimal Range	Hex Range
Signed	**-128 to 127**	**0x80 to 0x7F**

Addition

AL = 25H = 0010 0101
ADD AL, 13H = 0001 0011
- - - - - - - - - - - - - -
Result = 0011 1000 = 38H

CF: 0

PF: 0

AF: 0

ZF: 0

SF: 0

OF: 0

AL = 7FH = 0111 1111
ADD AL, 01H = 0000 0001
- - - - - - - - - - - - - -
Result = 1000 0000 = 80H

CF: 0

PF: 0

AF: 1

ZF: 0

SF: 1

OF: 1 (signed: +127 + 1 → -128 → sign flipped)

AL = 50H = 0101 0000
ADD AL, 50H = 0101 0000
- - - - - - - - - - - - - -
Result = 1010 0000 = A0H

CF: 0

PF: 1 (even parity)

AF: 0

ZF: 0

SF: 1

OF: 1 (signed: +80 + 80 = -96, invalid sign change)

AL = F0H = 1111 0000
ADD AL, 20H = 0010 0000
- - - - - - - - - - - - - -
Result = 0001 0000 = 10H

CF: 1

PF: 0

AF: 0

ZF: 0

SF: 0

OF: 0

Subtraction

AL = A0H = 1010 0000
SUB AL, 20H = 0010 0000

Result = 1000 0000 = 80H

CF: 0

PF: 0

AF: 0

ZF: 0

SF: 1

OF: 0

AL = 7FH = 0111 1111
SUB AL, FFH = 1111 1111

Result = 1000 0000 = 80H

CF: 1

PF: 0

AF: 1

ZF: 0

SF: 1

OF: 1 (signed: 127 - (-1) = 128 → overflow)

SUB AL, 10H

```
AL      = 3CH = 0011 1100
Operand = 10H = 0001 0000
---------------------------
Result  = 2CH = 0010 1100
```

🏳 **Flag Status:**

- **CF = 0**.
- **PF = 0**
- **AF = 0**
- **ZF = 0**
- **SF = 0**
- **OF = 0** → No signed overflow (positive - positive = positive).

AL = 80H = 1000 0000
SUB AL, 01H = 0000 0001
- - - - - - - - - - - - - -
Result = 0111 1111 = 7FH

CF: 0

PF: 0 (7 ones = odd → PF = 0)

AF: 1

ZF: 0

SF: 0

OF: 1 (signed: -128 – 1 = +127 → overflow)